SOLID Principles

Guidelines for Maintainable Object-Oriented Design

SOLID is a mnemonic for five design principles that help create software that's easier to maintain, extend, and understand. Robert C. Martin ("Uncle Bob") popularized them in the early 2000s.

These aren't laws—they're guidelines. Following them blindly can create unnecessary complexity. But violating them often leads to code that's hard to change.

S — Single Responsibility Principle

A class should have only one reason to change.

Each class should do one thing well. If a class handles both user authentication and email formatting, it has two reasons to change and two ways to break. Split responsibilities into separate classes.

This makes code easier to understand, test, and modify without unintended side effects.

O — Open/Closed Principle

Software entities should be open for extension but closed for modification.

You should be able to add new behavior without changing existing code. This usually means programming to abstractions—interfaces or base classes—so new implementations can be added without modifying the code that uses them.

When you find yourself repeatedly modifying a class to handle new cases, you're likely violating this principle.

L — Liskov Substitution Principle

Subtypes must be substitutable for their base types.

If code works with a base class, it should work correctly with any subclass. A `Square` that inherits from `Rectangle` but breaks when you set width and height independently violates this principle.

Violations indicate your inheritance hierarchy doesn't match reality. Consider composition instead.

I — Interface Segregation Principle

Clients should not be forced to depend on interfaces they don't use.

Don't create fat interfaces that force implementers to provide methods they don't need. If some users only need `read()` and others need `read()` and `write()`, split into separate interfaces.

Small, focused interfaces are easier to implement and less likely to force unnecessary changes.

D — Dependency Inversion Principle

Depend on abstractions, not concretions.

High-level modules shouldn't depend on low-level modules. Both should depend on abstractions. A business logic class shouldn't directly instantiate a database connection—it should receive an abstraction it can work with.

This decouples your code, making it easier to test and modify. You can swap implementations without changing the code that uses them.

References

- Robert C. Martin, *Clean Architecture* (2017) — Deep dive into SOLID and architectural principles.
- Robert C. Martin, "The Principles of OOD" — Original articles on SOLID principles.